

Facultad de Ingeniería

Manejo e implementación de archivos

Docente: Manuel Gustavo Rojas Vásquez



PROYECTO 1

Rodrigo Andrés Pinto Ticú

1507325

Quetzaltenango, 16 de septiembre del 2026

Contenido

Formato elegido para el almacenamiento de la configuración:	3
Implementación de la escritura y respaldo:.....	3
Manejo de errores:	6
Archivo generado:	7
Capturas de pantalla	7

Formato elegido para el almacenamiento de la configuración:

Para el almacenamiento de la configuración de mi sistema, decidí por utilizar un archivo .json para almacenar: el nombre del usuario, el tema escogido, lenguaje, tamaño de fuente, color del menú de fondo, color de fuente y la fotografía del usuario.

Entre mis opciones, elegí json debido a la facilidad para almacenar los datos por medio de claves, hago una comparación directa con un archivo .txt. Elegí .json en su mayoría debido a la fotografía del usuario y las entradas directas que él mismo puede manipular. Si se almacena la información de tal forma que cada línea almacene un dato diferente, podría ocurrir que el usuario coloque su nombre con un salto de línea “\n”, provocando que se guarde este salto de línea y mueva el resto de las configuraciones de lugar, provocando un error. Para solucionar esto, podría hacer que el nombre del usuario se almacene sin saltos de línea, pero el error persiste en la fotografía. Como esta se almacena en formato binario, ocasionalmente se crean caracteres del tipo “\n”, lo cuales no puedo eliminar debido a que estaría corrompiendo la secuencia de binarios del archivo.

Por las razones que anteriormente describí, fue que decidí utilizar json, al existir una clave única y una sintaxis que no tiene problemas con los saltos de línea, termina siendo mucho más legible y facilita el procesamiento de la información, al entregarlo directamente como un diccionario. Ante una corrupción, el archivo json me permite salvar las claves que no se corrompieron, algo que con txt sería más complicado debido a los saltos de línea. El archivo .json es más pesado que un archivo .txt, pero es más sencillo almacenar información con varios tipos de datos, no únicamente variables tipo texto.

Implementación de la escritura y respaldo:

Para la carga y el guardado/actualización del archivo de configuración, primero inicializo las rutas necesarias que voy a utilizar, las rutas son: ruta del archivo normal, ruta del archivo temporal y ruta del archivo backup

```
SETTINGS_PATH = ".data/config.json"
SETTINGS_PATH_TEMP = ".data/config_temp.json"
SETTINGS_PATH_BACKUP = ".data/config.bak"
```

Método load():

Para cargar la información, controlando cualquier excepción trato de abrir el contenido de SETTINGS_PATH, en caso exista cargo la información con ayuda de la librería json, si no se encuentra ningún dato corrupto, se procede a retornar la información dentro de la dataclass Settings; en caso no se encuentren datos dentro del archivo, se retorna una dataclass con valores por defecto

```

try:
    with open(SETTINGS_PATH, 'r', encoding='utf-8') as file:
        data = json.load(file)

    if data:
        return Settings(
            data["user_name"],
            data["theme"],
            data["lenguague"],
            data["font_size"],
            data["menu_color"],
            data["foreground"],
            data["photo"]
        )
    else:
        return Settings(
            "N/A",
            0,
            "es",
            15,
            "■ #F5F5F5",
            "□ #242424",
            ""
        )

```

Si existe algún problema/excepción en el proceso de cargar la información, se procede a utilizar archivo backup. Primero, se verifica si existe; en caso de que no, nuevamente se retorna los valores por defecto de la aplicación; si este existe, se crea un nuevo archivo config a manera de copia del archivo backup, para esta copia, se duplica el archivo cambiando su extensión .bak por .json; luego, se abre el nuevo archivo .json, como este tiene la información proveniente del archivo .bak, puede leer la información almacenada

```

except:
    if os.path.exists(SETTINGS_PATH_BACKUP):
        new_path = SETTINGS_PATH_BACKUP.replace('.bak', '.json')
        with open(SETTINGS_PATH_BACKUP, 'rb') as origin, open(new_path, 'wb') as new_:
            new_.write(origin.read())

        with open(new_path, "r", encoding="utf-8") as old_file:
            old = json.load(old_file)

        return Settings(
            old["user_name"],
            old["theme"],
            old["lenguague"],
            old["font_size"],
            old["menu_color"],
            old["foreground"],
            old["photo"]
        )
    else:
        return Settings(
            "N/A",
            0,
            "es",
            15,
            "■ #F5F5F5",
            "□ #242424",
            ""
        )

```

Método save():

Para el guardado de la información; primero, se almacena la configuración actual en la ruta del archivo temporal, en caso no exista ningún error al almacenar la información, se verifica si el archivo existe (se creó correctamente), si esto ocurre, se procede a crear el archivo .bak a partir de la información que fue guardada desde el archivo temporal, si todo ocurre sin inconvenientes, el archivo temporal sobrescribe al archivo config anterior, dejando para el final el cambio de nombre del archivo, pasando de "config_temp.json" a "config.json".

```
try:
    with open(SETTINGS_PATH_TEMP, 'w', encoding='utf-8') as file:
        json.dump(
            settings.setJson(),
            file,
            indent=4)
        #print(settings.setJson())

    if os.path.exists(SETTINGS_PATH_TEMP):
        with open(SETTINGS_PATH_TEMP, "r", encoding="utf-8") as old_file:
            old = json.load(old_file)
        with open(SETTINGS_PATH_BACKUP, "w", encoding="utf-8") as new_file:
            json.dump(old, new_file, indent=4, ensure_ascii=False)

        os.replace(SETTINGS_PATH_TEMP, SETTINGS_PATH)

    if os.path.exists(SETTINGS_PATH_TEMP):
        os.rename(SETTINGS_PATH_TEMP, SETTINGS_PATH)
```

En caso ocurra algún error en el guardado, únicamente se procede a guardar un archivo con la información por defecto, haciendo el mismo proceso de escribirlo primero en temporal y luego en cambiando el nombre para que quede como el archivo config.json.

```
except:
    with open(SETTINGS_PATH_TEMP, 'w', encoding='utf-8') as file:
        json.dump(
            Settings(
                "N/A",
                0,
                "es",
                15,
                "■ #F5F5F5",
                "□ #242424",
                ""
            ).setJson(),
            file,
            indent=4)
        if os.path.exists(SETTINGS_PATH_TEMP):
            os.rename(SETTINGS_PATH_TEMP, SETTINGS_PATH)
```

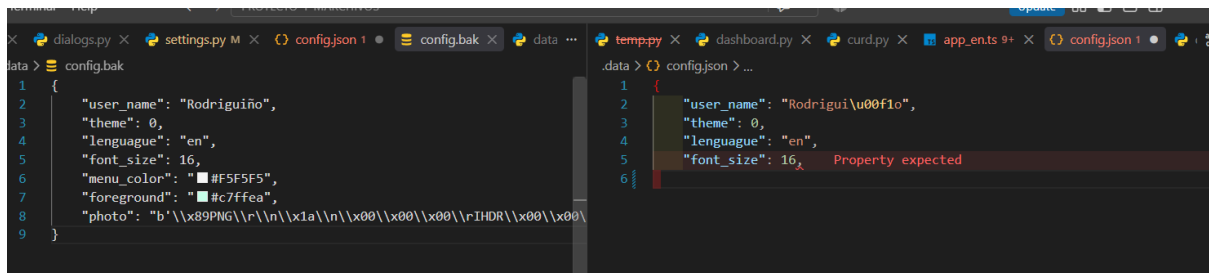
Manejo de errores:

Archivo ausente:

Si existe un archivo .bak, se crea el nuevo archivo config.json a partir de .bak (sin sobrescribir o reemplazar a .bak) y luego se carga su información. En caso que no exista un archivo .bak, se procede a generar un archivo con información por defecto.

Archivo corrupto:

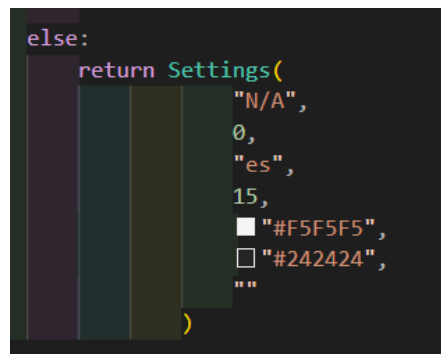
Si se encuentra que la información del archivo config.json no está completa, se recurre al archivo .bak para volver a cargar la información al igual que cuando el archivo está ausente.



MISMO CASO PARA AMBOS ERRORES

Sin permisos:

El sistema al no encontrar permisos para la manipulación de este archivo carga los valores predefinidos dentro del sistema y trata de generar el archivo de configuración, en caso este no tenga los permisos para generar los archivos, seguirá cargando únicamente los datos por defecto, hasta que el usuario otorgue los permisos para la manipulación de archivos.



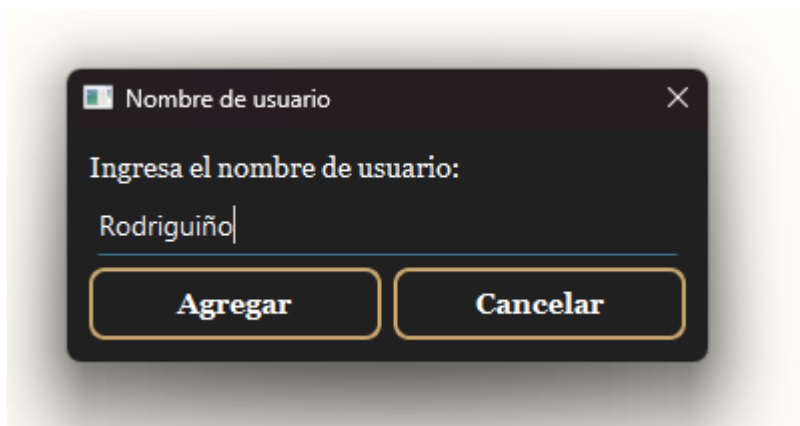
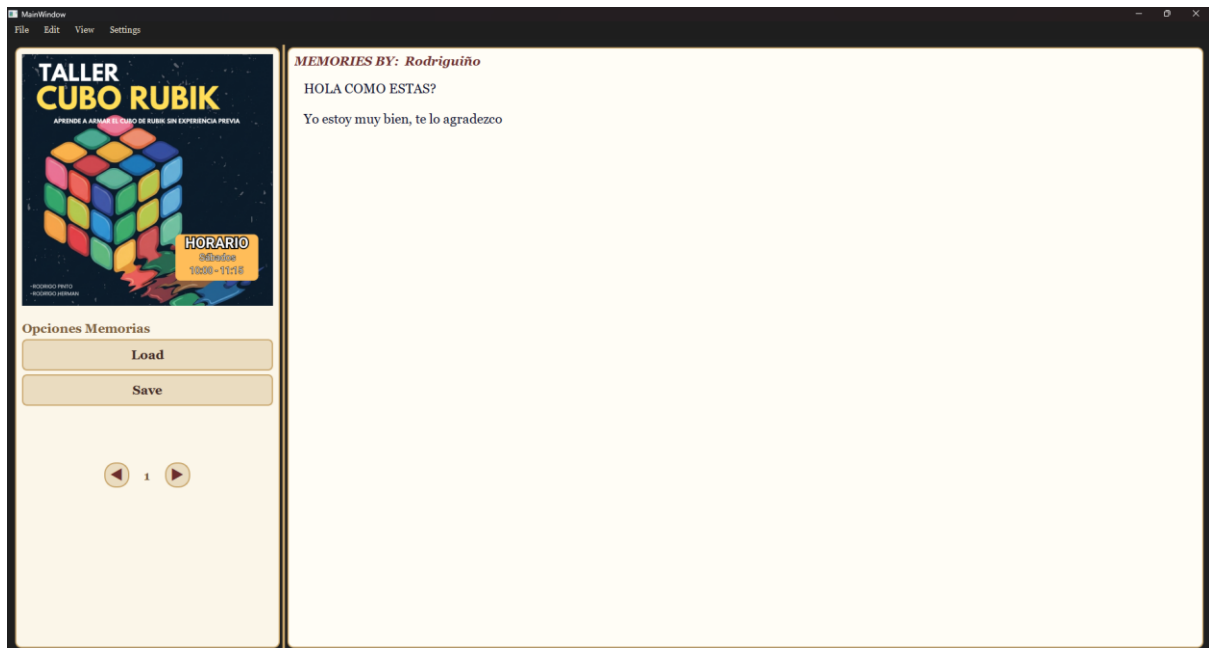
VALORES PREDEFINIDOS PARA EVITAR UN CIERRE O UN CRASH

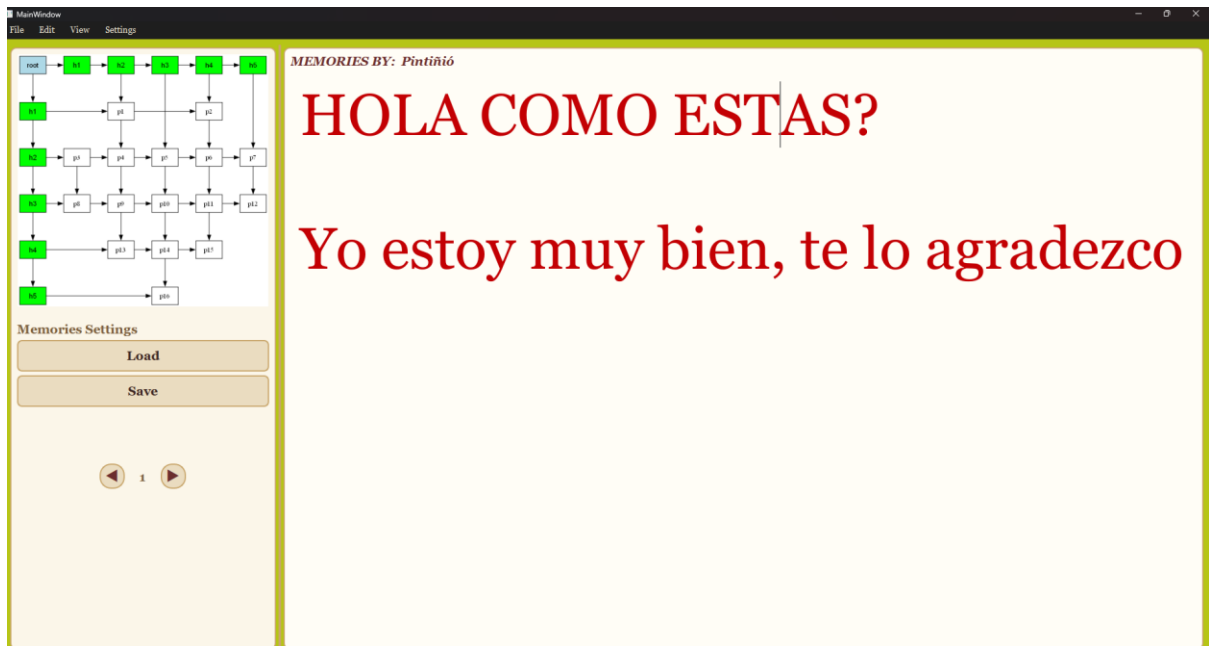
Archivo generado:

Archivo config.json

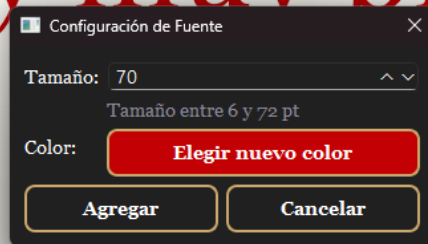
```
.data > config.json > ...
1  {
2    "user_name": "Rodríguez",
3    "theme": 0,
4    "language": "en",
5    "font_size": 16,
6    "menu_color": "#F5F5F5",
7    "foreground": "#C7FFEA",
8    "photo": "b'\\x89PNG\\r\\n\\x1a\\n\\x00\\x00\\x00\\rIHDR\\x00\\x00\\x03\\x84\\x00\\x00\\x03\\x84\\x08\\x02\\x00\\x00\\x00v,hI\\'
9  }
```

Capturas de pantalla:





o estoy muy bien, te lo



Yo estoy muy bien, te lo agradezco

